assignment3_part4

October 23, 2026

1 Assignment 3: Dealing with Vector and Matrix Real-World Data (Part IV)

1.1 Patterns in Matrix Data

In the last part of the assignment, let's explore the patterns in a matrix as a whole instead of in individual vectors. As discussed in class, one type of such patterns are *eigenvectors*, which can be obtained through *Singular Value Decomposition* (SVD):

$$X = U\Sigma V^T.$$

X is an $n \times p$ matrix, $U \cdot U^T = I$, $V \cdot V^T = I$, and Σ is an $n \times p$ diagonal matrix with non-zero singular values.

Let's first walk through the example in the lecture.

[35]:
```
import pandas as pd
import numpy as np
from sklearn.decomposition import TruncatedSVD
from scipy.sparse import csr_matrix

np.set_printoptions(precision=6)
np.set_printoptions(suppress=True)
```

[36]:
```
doc_word_df = pd.DataFrame([[1, 1, 1, 0, 0],
                            [2, 2, 2, 0, 0],
                            [1, 1, 1, 0, 0],
                            [5, 5, 5, 0, 0],
                            [0, 0, 0, 2, 2],
                            [0, 0, 0, 3, 3],
                            [0, 0, 0, 1, 1]])
doc_word_df.columns = ['data', 'information', 'retrieval', 'brain', 'lung']
doc_word_df.index = ['Data Science 1', 'Data Science 2', 'Data Science 3, ',␣
 ↪'Data Science 4',
                     'Medical Report 1', 'Medical Report 1', 'Medical Report 1']
doc_word_df
```

[36]:

	data	information	retrieval	brain	lung
Data Science 1	1	1	1	0	0
Data Science 2	2	2	2	0	0
Data Science 3,	1	1	1	0	0
Data Science 4	5	5	5	0	0
Medical Report 1	0	0	0	2	2
Medical Report 1	0	0	0	3	3
Medical Report 1	0	0	0	1	1

Several Python packages, such as NumPy, SciPy, and scikit-learn, all provide APIs for SVD. In this assignment, we will use `TruncatedSVD` API from scikit-learn, as it allows us to compute only the largest k singular values in Σ and the corresponding k columns (rows) of U (V^T), which is more efficient and practical in real-world applications. You can learn more about the API from its documentation.

For this example, we explicitly specify $k = 2$ (so we are only interested in the first 2 components), and we assign a constant to `random_state` to ensure that the result is reproducible.

[37]:
```
doc_word_svd = TruncatedSVD(n_components=2, random_state=0)
doc_word_svd.fit(doc_word_df)
```

[37]:
```
TruncatedSVD(random_state=0)
```

Now the object `doc_word_svd` stores all the necessary results of the decomposed matrix. You can view the singular values (the diagonal values in Σ) in the `singular_values_` attribute and the row vectors of V^T in the `components_` attribute.

[38]:
```
sigma_diag = doc_word_svd.singular_values_
v_t = doc_word_svd.components_
print(sigma_diag)
print(v_t)
```

```
[9.643651 5.291503]
[[ 0.57735   0.57735   0.57735  -0.       -0.      ]
 [ 0.        0.        0.        0.707107  0.707107]]
```

You may wonder where the U matrix is. Although U is not explicitly stored in the `TruncatedSVD` object. it can be recovered with X, Σ, and V, based on the following formula:

$$X^* = U\Sigma = XV.$$

Thus, U can be written as

$$U = X^*\Sigma^{-1} = XV\Sigma^{-1}.$$

We can verify this with the following code:

[39]:
```
x = doc_word_df.values
v = v_t.T
sigma_inverse = np.linalg.inv(np.diag(sigma_diag))

u = x.dot(v).dot(sigma_inverse)
print(u)
```

```
[[ 0.179605  0.      ]
 [ 0.359211  0.      ]
 [ 0.179605  0.      ]
 [ 0.898027  0.      ]
 [-0.        0.534522]
 [-0.        0.801784]
 [-0.        0.267261]]
```

Now let's try to interpret U, Σ, and V^T by thinking about the following questions (not graded):

- Which codes the Data Science / Medical concept of each document?
- Which codes the strength of each concept?
- Which codes the word representation of each concept?

One important application of SVD is to transform the original matrix X in to a new matrix X^*. This transformation projects the p-dimensional row vectors in the original matrix into k-dimensional vectors in the new vector space. X^* encodes most of the information in the original X matrix with fewer dimensions.

Mathamatically, the transformation is made by post-multiplying X with the V matrix ($ X^* = XV$). The `TruncatedSVD` API has implemented such transformation as a `transform` function. You may verify this with the following code:

[40]:
```
print(doc_word_svd.transform(doc_word_df))
```

```
[[ 1.732051  0.      ]
 [ 3.464102  0.      ]
 [ 1.732051  0.      ]
 [ 8.660254  0.      ]
 [-0.        2.828427]
 [-0.        4.242641]
 [-0.        1.414214]]
```

[41]:
```
print(doc_word_df.values.dot(v))
```

```
[[ 1.732051  0.      ]
 [ 3.464102  0.      ]
 [ 1.732051  0.      ]
 [ 8.660254  0.      ]
 [-0.        2.828427]
 [-0.        4.242641]
 [-0.        1.414214]]
```

Now that you are familiar with the SVD operations. Let's apply it to the Montréal restaurant dataset. Run the following code block to load the dataset prepared in Part I of this assignment.

[55]:
```
business_df = pd.read_csv('assets/montreal_business.csv')
business_df.set_index('business_id', inplace=True)

review_df = pd.read_csv('assets/montreal_user.csv')
rating_df = review_df.pivot_table(index=['business_id'], columns=['user_id'],␣
 ↪values='stars')
rating_df.fillna(0, inplace=True)

missing_business_id = set(business_df.index) - set(rating_df.index)
business_df.drop(missing_business_id, inplace=True)
```

[67]:
```
print(business_df.shape)
business_df.head()
```

```
(2770, 4)
```

[67]:

business_id	name	categories	stars	postal_code
-1xuC540Nycht_iWFeJ-dw	Romados	Restaurants, Portuguese, Bakeries, Food, Chick…	4.5	H2W 1C8
-7bRnaHp7OHz8KW-THqP4w	Buffalo Bill Wings	Restaurants, Chicken Wings	1.5	H4C 1G7
-92cC6-X87HQ1DE1UHOx3w	Junior	Pan Asian, Restaurants, Asian Fusion, Filipino	3.5	H3J 1M8
-AgfhwHOYrsPKt-_xV_Ipg	Soupesoup	Sandwiches, Restaurants, Cafes, Soup	4.0	H2W 1G8
-FDkvLmwaBrtVgYFqEWeWA	Café Juicy Lotus	Vegetarian, Food, Salad, Restaurants, Organic …	3.5	H4A 1C9

1.1.1 Exercise 5. (15 pts.)

Complete the following `transformed_rating` function, which transforms the original $n \times p$ rating matrix into a new $n \times k$ matrix using the `TruncatedSVD` API. The function should return the $n \times k$ matrix.

Note: 1. Please set the `random_state` to 0 so that the results do not change over different runs. 2. k can take values between 1 and p, not necessarily the 2 used in the document-word matrix example. 3. You may either use the `transform` function in the `TruncatedSVD` object or use the dot product. Both are OK. 4. We convert `original_matrix` to a sparse matrix for you in order to save time in the grading process. You do not need to make any accomodation for this in your solution.

[43]:
```
def svd_transformed_rating(original_matrix, k, random_state=0):

    transformed_matrix = None
    original_matrix = csr_matrix(original_matrix)

    # YOUR CODE HERE
    # raise NotImplementedError()
    np.random.seed(random_state)
    #transformed_matrix = TruncatedSVD(n_components=k, random_state=0).
 ↪fit_transform(original_matrix)

    tsvd = TruncatedSVD(n_components=k, random_state=0).fit(original_matrix)
    transformed_matrix = original_matrix.dot(tsvd.components_.T)

    return transformed_matrix
```

With the `svd_transformed_rating` function, we can calculate the transformed matrix through the following command.

[44]:
```
rating_matrix_d2 = svd_transformed_rating(rating_df, 2)
# You can uncomment the following line to view the result
# rating_matrix_d2
```

[45]:
```
# This code block tests if the `svd_transformed_rating` function is implemented␣
 ↪correctly.
# We hide some tests, so passing all the displayed assertions does not␣
 ↪guarantee full points.

rating_matrix_d2 = svd_transformed_rating(rating_df, 2)
# d2_groundtruth is calculated via the following command:
# d2_groundtruth = svd_transformed_rating(rating_df, 4)[:4]
d2_groundtruth = np.array([[26.830835356   , -6.960033590895],
                           [ 0.240134782174,  0.031972398386],
                           [ 2.971341010574, -2.141467087847],
                           [ 0.997482022276, -0.599895804555],
                           [ 0.470445852682, -0.482343299048]])
assert np.allclose(rating_matrix_d2[:5], d2_groundtruth)

rating_matrix_d5 = svd_transformed_rating(rating_df, 5)
d5_groundtruth = np.array([[26.828890919511, -6.950258673617,  1.515366172479,␣
 ↪-3.295739836372,  4.157186247066],
```

```
                          [ 0.24012936551 ,  0.031884190966, -0.018132185754,␣
 ↪-0.166903622477, -0.028071803934],
                          [ 2.9713046863  , -2.141818808721, -1.232252381363,␣
 ↪-0.691324426529,  0.11961284882 ],
                          [ 0.997484724453, -0.59970688578 , -0.107441873278, ␣
 ↪0.047329773205,  0.160751420569],
                          [ 0.470427947552, -0.482204394535,  0.062480138124,␣
 ↪-0.238280610506,  0.055425249737]])
assert np.allclose(rating_matrix_d5[:5], d5_groundtruth)

rating_matrix_transformed = svd_transformed_rating(rating_df, 100)
assert rating_matrix_transformed.shape == (2770,100), "The transformed matrix␣
 ↪is of the wrong dimension."
for i in range(100):
    for j in range(i):
        assert abs(rating_matrix_transformed.T[i].dot(rating_matrix_transformed.
 ↪T[j])) < 1e-8, \
            "The column vectors in X* should be orthogonal."
```

A quick note on the above test cell: Ideally, the first k' columns should stay the same for any k as long as $k > k'$. However, you may notice small differences in the test cell (e.g. 26.830835356 vs. 26.828890919511). This is because the `TruncatedSVD` API uses a randomized SVD solver to speed up the calculation, so the output may lose minor precision.

As you can see, we even increased k to 100, which is much larger than $k = 2$ used in the document-word example. However, this is still much smaller than the 11,937 dimensions in the original rating matrix, and we shall soon see that these 100 dimensions have preserved most of the information in the original matrix.

1.1.2 Exercise 6. (5 pts).

This exercise is to help you examine the power of SVD. That is, we want to see how the 100-dimension vectors preserve much information from the 11,937-dimensional vectors. Indeed, we shall see that the similarity between vectors is preserved after the SVD transformation. To show that, we are going to combine several techniques we have learned so far, including vector similarity, patterns in matrix data, and itemset similarity.

For your first task, complete the `find_max_dot_prod_restaurants` and the `find_max_dot_prod_restaurants_transformed` function. Each function returns the `business_id` of the top-n restaurants that have the largest dot product with Modavie. The `find_max_dot_prod_restaurants` function calculates the dot product based on the raw vectors (11,937 dimensions). The `find_max_dot_prod_restaurants_transformed` function calculates with the transformed vectors (100 dimensions). You may copy and paste `find_max_dot_prod_restaurants` from Part III of this assignment.

We have provided all the test cases to help you verify your results.

Hints

- For finding the top modavie dot products, you may find your solution to part 3 of this assignment useful.

[46]:
```
# This cell helps you prepare a new rating dataframe using the transformed␣
 ↪vectors.
rating_matrix_transformed = svd_transformed_rating(rating_df, 100)
rating_df_transformed = pd.DataFrame(rating_matrix_transformed, index=rating_df.
 ↪index)
rating_df_transformed.head()
```

[46]:

business_id	0	1	2	3	4	\
-1xuC540Nycht_iWFeJ-dw	26.829882	-6.949084	1.637419	-3.495390	4.253959	
-7bRnaHp7OHz8KW-THqP4w	0.240134	0.031948	-0.017022	-0.169980	-0.026894	
-92cC6-X87HQ1DE1UHOx3w	2.971325	-2.141960	-1.232110	-0.698304	0.111484	
-AgfhwHOYrsPKt-_xV_Ipg	0.997471	-0.599784	-0.108910	0.053457	0.161304	
-FDkvLmwaBrtVgYFqEWeWA	0.470432	-0.481998	0.060954	-0.237180	0.047426	

business_id	5	6	7	8	9	\
-1xuC540Nycht_iWFeJ-dw	-1.578638	-4.723778	39.660973	-18.228902	23.421889	
-7bRnaHp7OHz8KW-THqP4w	-0.095219	-0.032490	0.103308	-0.170128	-0.030196	
-92cC6-X87HQ1DE1UHOx3w	-0.469199	0.181528	-1.522476	0.333825	0.420363	
-AgfhwHOYrsPKt-_xV_Ipg	0.202928	-0.815409	0.270971	0.649032	0.253430	
-FDkvLmwaBrtVgYFqEWeWA	0.081098	-0.075756	-0.257721	0.141577	0.436627	

business_id	…	90	91	92	93	94	\
-1xuC540Nycht_iWFeJ-dw	…	0.767882	1.533552	1.182358	-0.950001	0.017549	
-7bRnaHp7OHz8KW-THqP4w	…	0.225307	0.001314	-0.318106	-0.128878	0.100352	
-92cC6-X87HQ1DE1UHOx3w	…	1.406269	1.265186	-1.808670	-0.257573	0.417470	
-AgfhwHOYrsPKt-_xV_Ipg	…	0.384626	-1.344930	0.325934	-0.000308	0.328897	
-FDkvLmwaBrtVgYFqEWeWA	…	-0.100854	-0.031623	-0.198573	0.596779	-0.659795	

business_id	95	96	97	98	99
-1xuC540Nycht_iWFeJ-dw	0.262306	-1.707314	-1.132702	-0.558816	0.669832
-7bRnaHp7OHz8KW-THqP4w	0.165589	0.204660	0.075091	0.270025	-0.273088
-92cC6-X87HQ1DE1UHOx3w	-0.545357	1.153764	0.511001	-0.184996	-1.439649
-AgfhwHOYrsPKt-_xV_Ipg	0.074603	0.192419	0.501441	-0.059467	-0.565910
-FDkvLmwaBrtVgYFqEWeWA	0.022381	-0.060685	0.787100	-0.165884	0.032865

[5 rows x 100 columns]

[68]:
```
print(rating_df.shape, rating_df_transformed.shape)
```

```
(2770, 11937) (2770, 100)
```

[77]:
```
def find_max_dot_prod_restaurants(top_n):
    modavie_id = business_df[business_df.name.str.contains("Modavie")].index[0]
    modavie_vector = rating_df.loc[modavie_id]
    # YOUR CODE HERE
    #raise NotImplementedError()
    business_df['modavie_dot_prod'] = rating_df.apply(lambda x:np.
 ↪dot(modavie_vector, x), axis=1)
    return business_df.sort_values('modavie_dot_prod', ascending=False).
 ↪drop(modavie_id).head(top_n)#['name'].tolist()

def find_max_dot_prod_restaurants_transformed(top_n):
    modavie_id = business_df[business_df.name.str.contains("Modavie")].index[0]
    modavie_vector_transformed = rating_df_transformed.loc[modavie_id]
    # YOUR CODE HERE
    #raise NotImplementedError()
    #print(modavie_vector_transformed.shape, business_df.shape)
    business_df['modavie_dot_prod'] = rating_df_transformed.apply(lambda x:np.
 ↪dot(modavie_vector_transformed, x), axis=1)
    return business_df.sort_values('modavie_dot_prod', ascending=False).
 ↪drop(modavie_id).head(top_n)#['name'].tolist()

#find_max_dot_prod_restaurants_transformed(10)
```

[78]:
```
# This code block help you verify if␣
 ↪`find_max_dot_prod_restaurants_transformed` is implemented correctly.
answer = find_max_dot_prod_restaurants_transformed(10)
assert answer.iloc[0]['name'] == "Schwartz's"
assert answer.iloc[1]['name'] == "La Banquise"
assert answer.iloc[2]['name'] == "Olive & Gourmando"
assert answer.iloc[3]['name'] == "Maison Christian Faure"
assert answer.iloc[4]['name'] == "Reuben's Deli & Steakhouse"

assert answer.iloc[5]['name'] == "Kazu"
assert answer.iloc[6]['name'] == "Belon"
assert answer.iloc[7]['name'] == "Au Pied de Cochon"
assert answer.iloc[8]['name'] == "L'Avenue"
assert answer.iloc[9]['name'] == "Poutineville"
```

1.1.3 Exercise 7. (5 pts).

Finally, complete the `jaccard_similarity_before_after_svd_transform` function, which calculates the Jaccard similarity between the top-n most similar restaurants (of course, this is an itemset!) before and after the SVD transformation.

** Hints **

- Refer back to your solution for assignment 2 part 3 if you've forgotten how to accomplish this.

[79]:
```
def jaccard_similarity_before_after_svd_transform(top_n):
    max_sim_restaurants = find_max_dot_prod_restaurants(top_n)
    max_sim_restaurants_transformed = find_max_dot_prod_restaurants_transformed(top_n)
    # construct the set of the names of the most similar restaurants
    business_id_before = set(max_sim_restaurants.name)
    business_id_after = set(max_sim_restaurants_transformed.name)
    # compute the Jaccard similarity between the two sets and return the Jaccard similarity
    # YOUR CODE HERE
    #raise NotImplementedError()
    jaccard_similarity = len(business_id_before.intersection(business_id_after)) / len(business_id_before.union(business_id_after))

    return jaccard_similarity
```

[80]:
```
# This code block checks `find_max_dot_prod_restaurants`, `find_max_dot_prod_restaurants_transformed`,
# and 'jaccard_similarity_before_after_svd_transform'. You should get the correct answer if all three
# functions are implemented correctly.
# We hide some tests, so passing all the displayed assertions does not guarantee full points.

assert abs(jaccard_similarity_before_after_svd_transform(5) - 1) < 1e-8
assert (abs(jaccard_similarity_before_after_svd_transform(10) - 9 / 11) < 1e-8) or (abs(jaccard_similarity_before_after_svd_transform(10) - 2 / 3) < 1e-8)
assert abs(jaccard_similarity_before_after_svd_transform(2770) - 1) < 1e-8
```

As you can see, you can perform many tasks on the transformed vectors just as what you can do on the raw vectors. Since the transformed vectors have far fewer dimensions, the efficiency is much higher with the transformed vectors. Beyond efficiency, dimension reduction has many other benefits for dealing with matrix data. You will learn these in the downstream machine learning classes, but let's stop here for now. Matrix analysis has lots of applications in recommender systems. We will revisit some of these techniques when that topic comes up.

This concludes this assignment.